



Applied ML

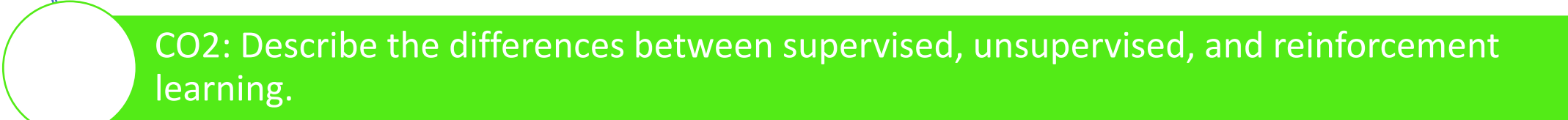
Prerequisite(s):	Elements of AI
------------------	----------------

Total Units to be Covered: 6	Total Contact Hours: 48
------------------------------	-------------------------

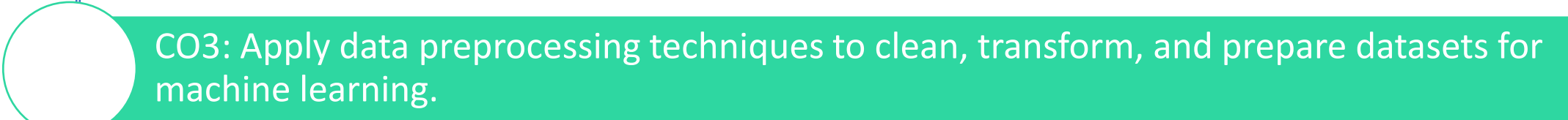
Course Outcomes



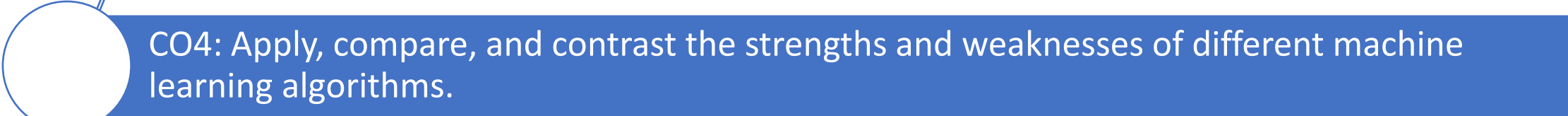
CO1: Recall and define key machine learning concepts, terminologies, and algorithms.



CO2: Describe the differences between supervised, unsupervised, and reinforcement learning.



CO3: Apply data preprocessing techniques to clean, transform, and prepare datasets for machine learning.



CO4: Apply, compare, and contrast the strengths and weaknesses of different machine learning algorithms.

Unit III: Optimizer function

Stochastic gradient descent, Mini-Batch Gradient Descent, Momentum, Adaptive gradient algorithm (Adagrad), Adam (Adaptive Moment Estimation), RMSprop (Root Mean Square Propagation), Adadelata

Why Do We Need Gradient Descent?

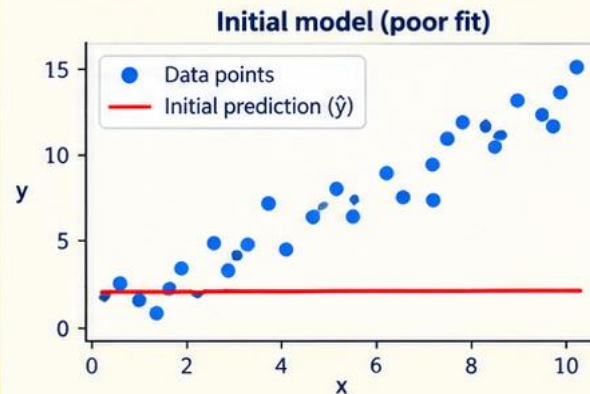
An iterative method to find the best model parameters by minimising prediction error.

The Problem

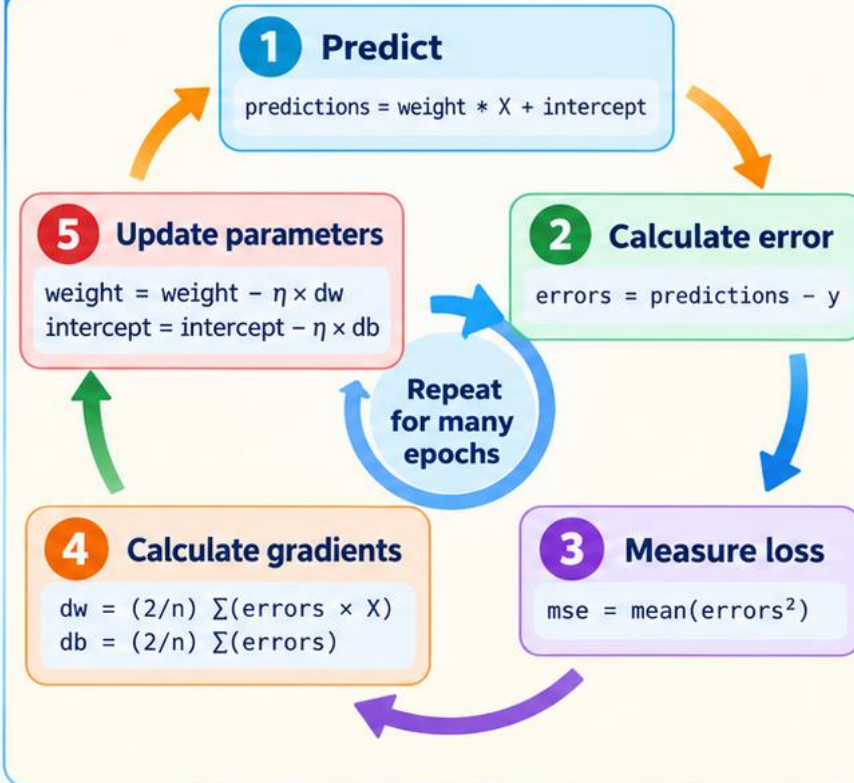
Model: $\hat{y} = wx + b$

At the beginning:
weight = 0.0
intercept = 0.0

Predictions are inaccurate
and MSE is high.

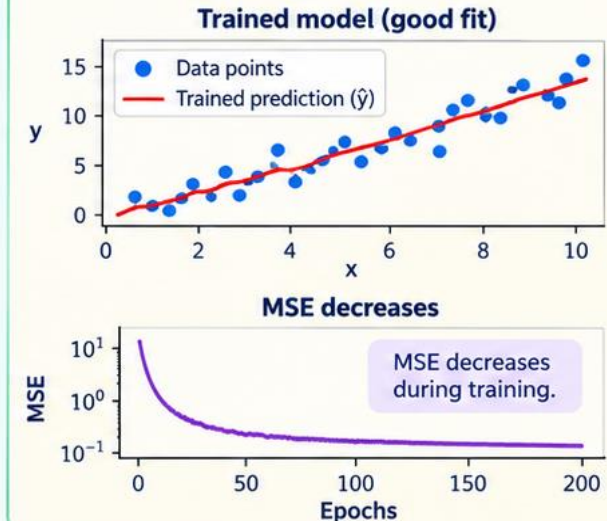


Gradient Descent Learning Loop



The Result

- ✓ Lower prediction error
- ✓ Better weight and intercept
- ✓ Regression line fits the data
- ✓ Model can predict new values



Gradient descent automatically finds parameter values that minimise prediction error:

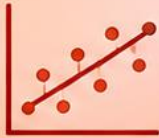
Useful for large datasets • many features • neural networks • models without a simple direct solution



Linear Regression

Model a continuous target as a linear combination of features.

Without regularisation



May overfit when features are many or highly correlated.

With regularisation



Add a penalty to the loss function to constrain model coefficients and reduce overfitting.

Ridge Regression — L2



L2

Shrinks coefficients; usually keeps all features.

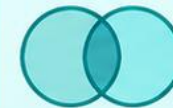
Lasso Regression — L1



L1

Can reduce some coefficients to zero; performs feature selection.

Elastic Net — L1 + L2



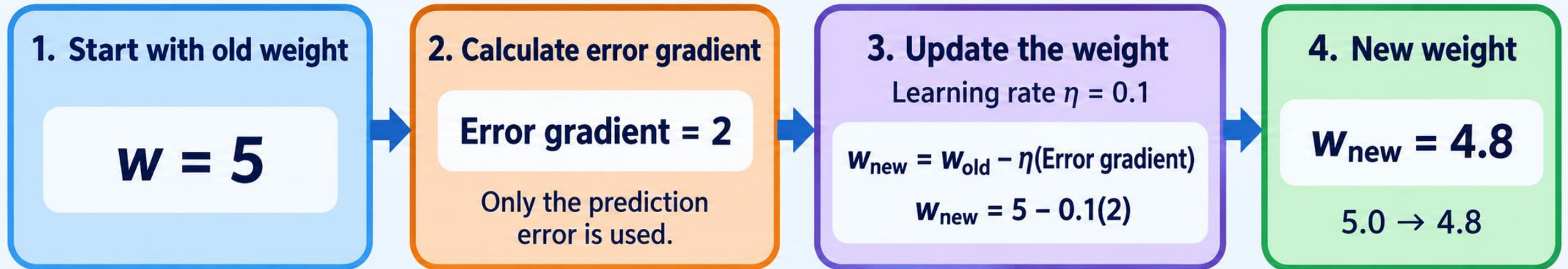
L1 + L2

Combines sparsity of Lasso with stability of Ridge.



Regularisation controls model complexity and helps improve generalisation.

Gradient Descent Without Regularisation



Gradient Descent Without Regularisation: Moving Down the Loss Curve





Takeaway: Gradient descent repeatedly moves the weight in the direction that reduces prediction error.



Gradient descent updates model parameters using only the training set.



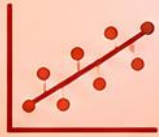
Validation helps make decisions. Test data checks the final model.



Linear Regression

Model a continuous target as a linear combination of features.

Without regularisation



May overfit when features are many or highly correlated.

With regularisation



Add a penalty to the loss function to constrain model coefficients and reduce overfitting.

Ridge Regression — L2



L2

Shrinks coefficients; usually keeps all features.

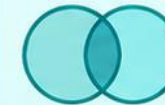
Lasso Regression — L1



L1

Can reduce some coefficients to zero; performs feature selection.

Elastic Net — L1 + L2



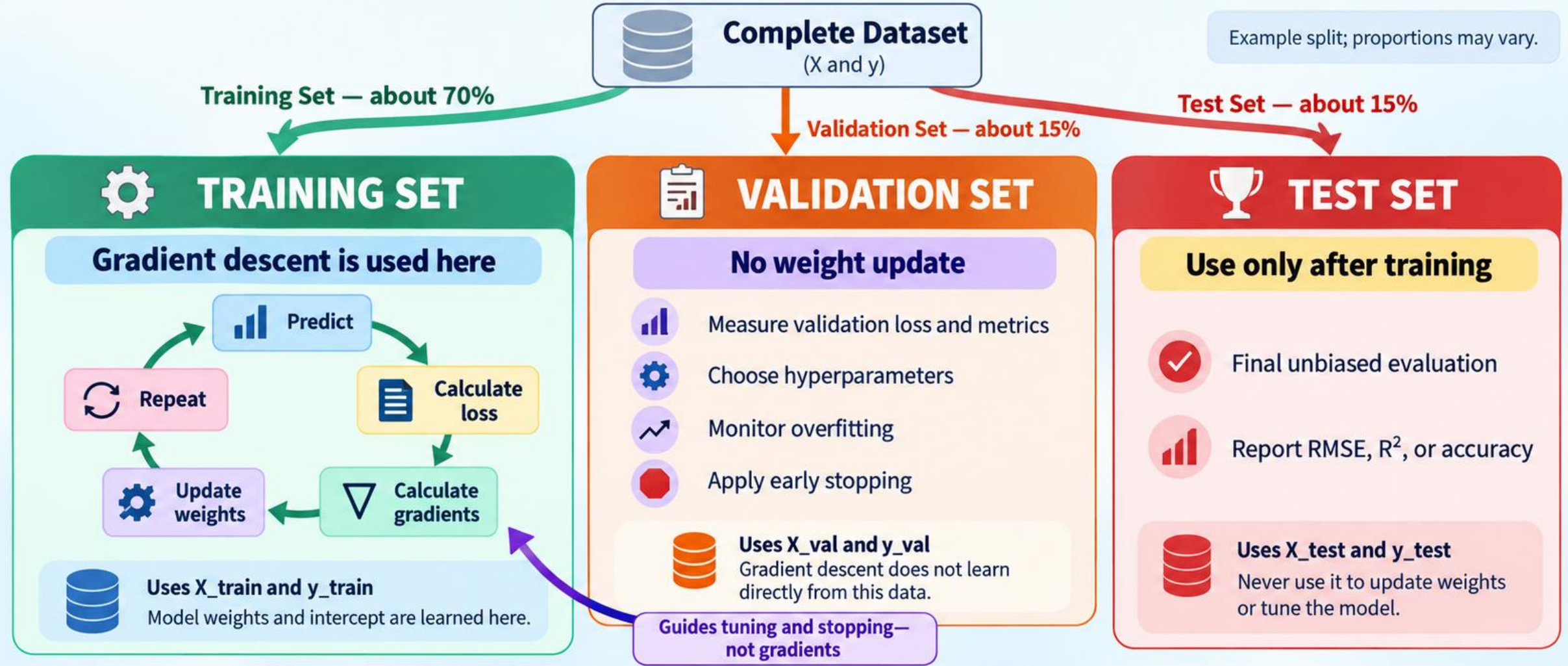
L1 + L2

Combines sparsity of Lasso with stability of Ridge.



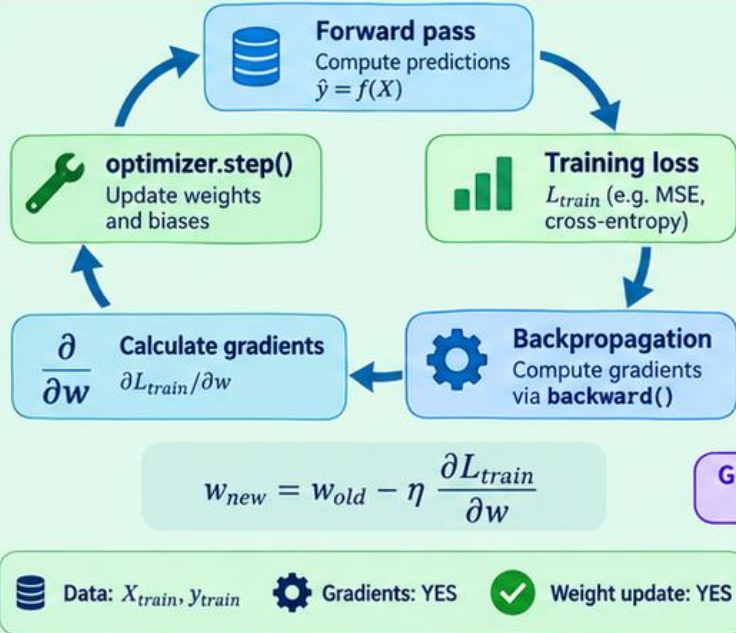
Regularisation controls model complexity and helps improve generalisation.

Where Is Gradient Descent Used in Data Partitioning?



1. TRAINING — Optimizer Works Here

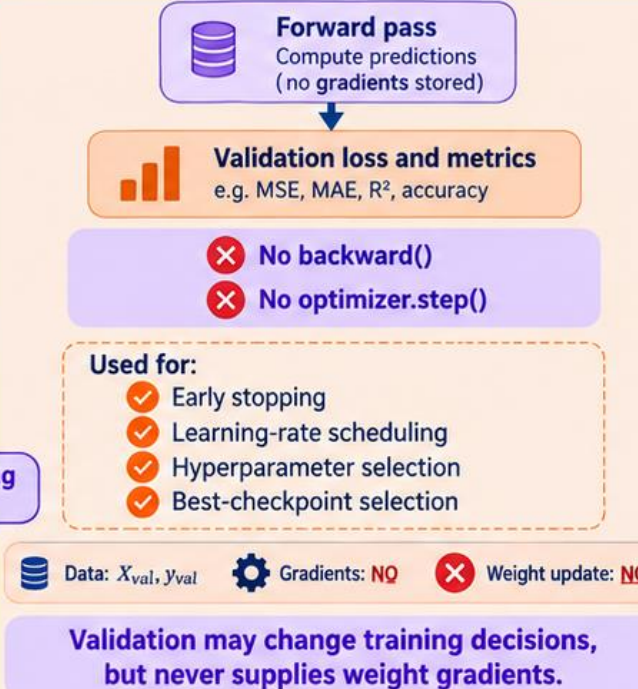
The optimizer uses gradients from the training loss to update model parameters.



Guides training indirectly

2. VALIDATION — Check and Guide

Monitor performance on unseen validation data to guide training decisions.



3. TESTING — Final Evaluation

Evaluate the selected final model on completely unseen test data.



Aspect	Training	Validation	Test
Forward pass	YES	YES	YES
Calculate gradients	YES	NO	NO
Optimizer updates weights	YES	NO	NO
Model decisions	Learn parameters	Tune and stop	Final report



Using validation or test data to update weights causes data leakage.



Training data teaches the model. Validation data guides the process. Test data evaluates the final model.



Hyperparameters of Gradient Descent

Small choices.
Big learning.



Learning Rate (η)

Controls the size of each weight update.

Too small: slow |
Too large: may overshoot

Examples: 0.1, 0.01, 0.001



Epochs

Maximum number of complete passes through the data.

Too few: underfitting |
Too many: wasted training

Examples: 100, 500, 1000



Batch Size

Number of samples used for one weight update.

1 = SGD | 32–256 = Mini-batch |
All = Batch GD



Learning-Rate Schedule

Changes the learning rate during training.

Types: constant, inverse scaling, adaptive



Tolerance

Minimum improvement required to continue.

Smaller value allows more precise convergence.

Examples: 10^{-3} , 10^{-4}



Patience

Number of epochs allowed without improvement.

Used for early stopping.

Examples: 5, 10, 20



Regularisation Strength (α)

Controls the penalty on large weights.

Too small: weak control |
Too large: underfitting

Examples: 0.0001, 0.001, 0.01



Momentum

Uses previous updates to accelerate learning.

Helps reduce oscillation.

Examples: 0.9, 0.99



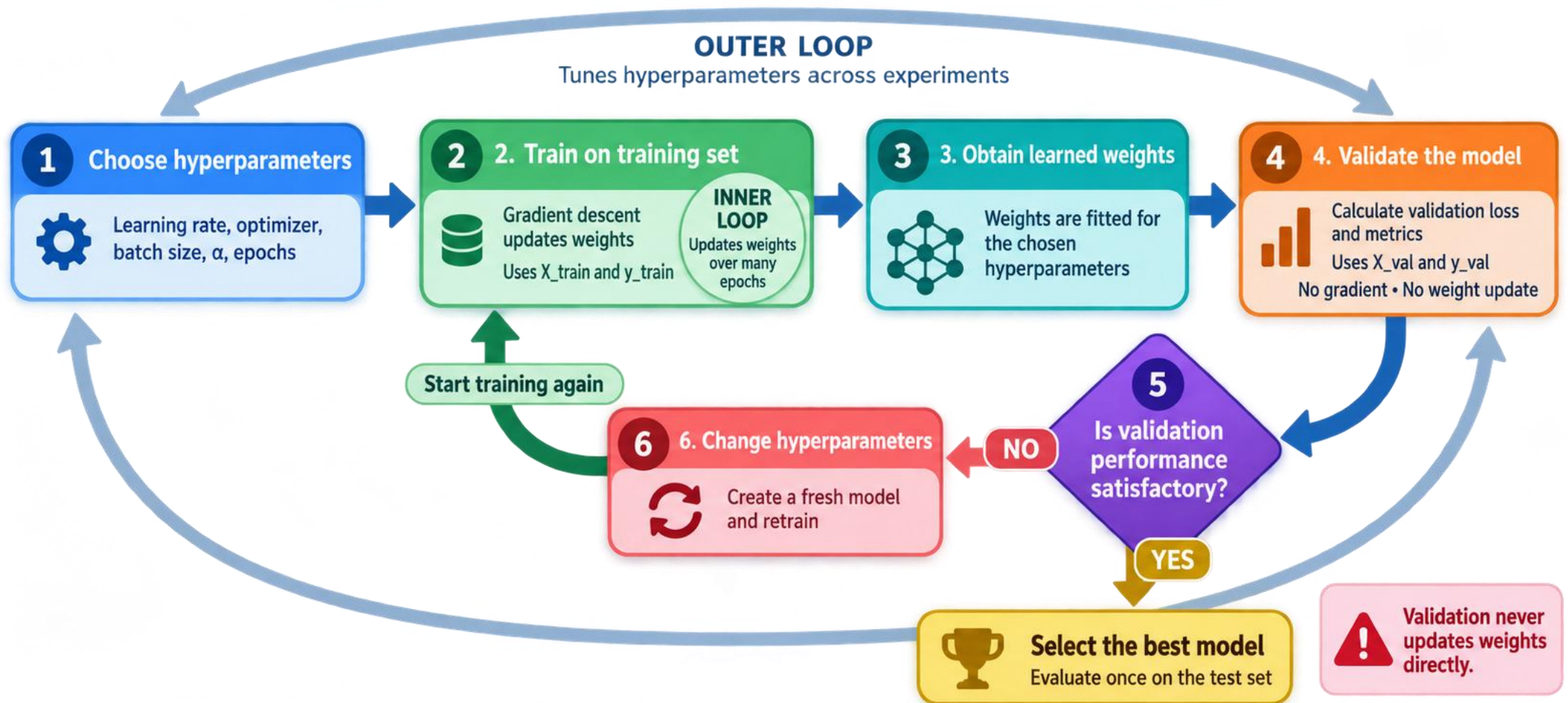
Weight update:

$$w_{new} = w_{old} - \eta \times \text{gradient}$$



The most influential choices are learning rate, epochs, batch size, and regularisation strength.

Training-Validation Hyperparameter Loop



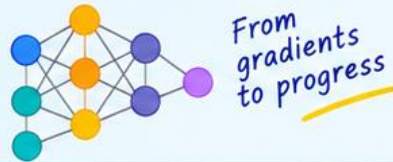
Training learns weights. Validation evaluates choices. Poor validation results lead to new hyperparameters and retraining.

OPTIMIZERS USED FOR TRAINING

How neural networks update weights to reduce loss



Better models brighter futures



1 STOCHASTIC GRADIENT DESCENT (SGD)

How it works: Updates weights using ONE randomly selected training example.

Update: $w \leftarrow w - \eta \nabla L_i(w)$

Advantages: Fast updates • Low memory • Can escape shallow minima

Limitations: Noisy path • Loss fluctuates • May converge slowly

Best for: Very large or streaming datasets



2 MINI-BATCH GRADIENT DESCENT

How it works: Updates weights using a small batch, commonly 32–256 examples.

Update: $w \leftarrow w - \eta (\text{average batch gradient})$

Advantages: Stable and efficient • GPU-friendly • Most widely used

Limitations: Batch size must be selected • Still has some noise

Best for: Most deep-learning training



3 MOMENTUM

How it works: Adds a fraction of the previous update to build speed.

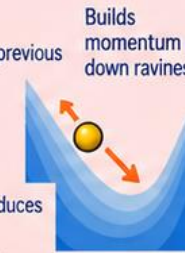
$$v_t \leftarrow \beta v_{t-1} + \nabla L(w_t)$$

$$w_{t+1} \leftarrow w_t - \eta v_t$$

Advantages: Faster convergence • Reduces zig-zagging

Limitations: Extra β hyperparameter • May overshoot

Best for: Ravines and consistent gradient directions



4 ADAGRAD

How it works: Gives each parameter its own learning rate; frequent features receive smaller steps.

Advantages: Excellent for sparse data • Automatic learning-rate adaptation

Limitations: Learning rate continually shrinks and may become nearly zero

Best for: Sparse features, text, and recommendation systems



5 RMSprop

How it works: Divides the gradient by a moving average of recent squared gradients.

Advantages: Prevents Adagrad's rapid decay • Handles changing gradients

Limitations: Requires decay-rate tuning

Best for: Recurrent networks and non-stationary objectives



6 ADADELTA

How it works: Uses moving windows of squared gradients and squared updates.

Advantages: Reduces need for a manually chosen learning rate • Robust scaling

Limitations: Sometimes slower than Adam • Less commonly used today

Best for: Problems sensitive to learning-rate scale



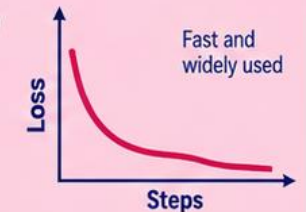
7 ADAM – ADAPTIVE MOMENT ESTIMATION

How it works: Combines Momentum (first moment) and RMSprop-like scaling (second moment).

Advantages: Fast • Adaptive • Works well with noisy or sparse gradients

Limitations: More memory • Can generalize worse than SGD in some tasks

Best for: Strong default choice for deep learning



QUICK COMPARISON

SGD
noisy, simple

Mini-Batch
balanced, standard

Momentum
gains speed

Adagrad
sparse-data specialist

RMSprop
controls squared gradients

Adadelta
scale-adaptive

Adam
momentum + adaptive rates

η = learning rate • β = momentum/decay factor • ∇L = loss gradient
Optimizers operate during TRAINING. Validation evaluates hyperparameters; it does not update weights.

Same idea. Brighter possibilities.

1 STOCHASTIC GRADIENT DESCENT (SGD)

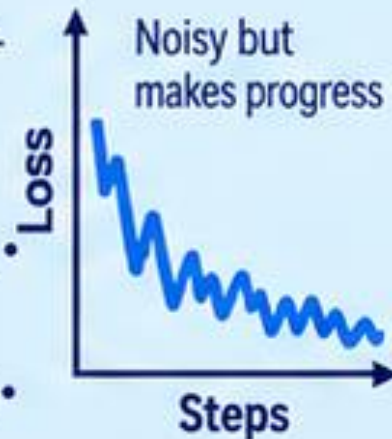
How it works: Updates weights using ONE randomly selected training example.

Update: $w \leftarrow w - \eta \nabla L_i(w)$

Advantages: Fast updates • Low memory • Can escape shallow minima

Limitations: Noisy path • Loss fluctuates • May converge slowly

Best for: Very large or streaming datasets



2 MINI-BATCH GRADIENT DESCENT

How it works: Updates weights using a small batch, commonly 32–256 examples.

Update: $w \leftarrow w - \eta (\text{average batch gradient})$

Advantages: Stable and efficient • GPU-friendly • Most widely used

Limitations: Batch size must be selected • Still has some noise

Best for: Most deep-learning training



3 MOMENTUM

How it works: Adds a fraction of the previous update to build speed.

$$v_t \leftarrow \beta v_{t-1} + \nabla L(w_t)$$

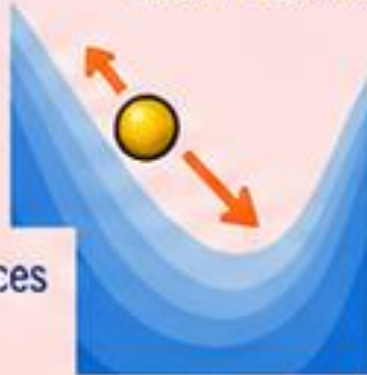
$$w_{t+1} \leftarrow w_t - \eta v_t$$

Advantages: Faster convergence • Reduces zig-zagging

Limitations: Extra β hyperparameter • May overshoot

Best for: Ravines and consistent gradient directions

Builds momentum down ravines



4 ADAGRAD

How it works: Gives each parameter its own learning rate; frequent features receive smaller steps.

Advantages: Excellent for sparse data • Automatic learning-rate adaptation

Limitations: Learning rate continually shrinks and may become nearly zero

Best for: Sparse features, text, and recommendation systems



5 RMSprop

How it works: Divides the gradient by a moving average of recent squared gradients.

Advantages: Prevents Adagrad's rapid decay • Handles changing gradients

Limitations: Requires decay-rate tuning

Best for: Recurrent networks and non-stationary objectives



6 ADADELTA

How it works: Uses moving windows of squared gradients and squared updates.

Advantages: Reduces need for a manually chosen learning rate • Robust scaling

Limitations: Sometimes slower than Adam • Less commonly used today

Best for: Problems sensitive to learning-rate scale



Adapts using moving windows

7 ADAM – ADAPTIVE MOMENT ESTIMATION

How it works: Combines Momentum (first moment) and RMSprop-like scaling (second moment).

Advantages: Fast • Adaptive • Works well with noisy or sparse gradients

Limitations: More memory • Can generalize worse than SGD in some tasks

Best for: Strong default choice for deep learning



Thank You